

Technical Interview Study Guide

Contents

1. JavaScript and coding habits	2
2. Node.js, HTTP, databases, and React	3
3. DSA patterns before the problems	5
4. Fifteen coding problems with explanations	6
5. Redis: fast reads without losing correctness	12
6. AWS: scheduled work that survives failure	13
7. RAG: an open-book answer with evidence	15
8. System design: a movie-discovery service	18
9. Project stories: turn the résumé into evidence	20
10. Three timed mock interviews for a friend to run	22
11. Quick recall cards	25
12. Optional videos and final practice	26

JavaScript • Node.js • DSA • Redis • AWS • RAG • system design • project stories

Prepared as a portable companion to [Interview Studio](#). This guide is for reading, annotating, and practicing aloud. The product design and sample data below are **hypothetical exercises**. Public descriptions of Shownex's movie-discovery features provide context; nothing here claims to describe its private systems or its actual interview questions.

How to use this guide

Read one chapter, close it, and explain the idea to a friend in your own words. For coding problems, attempt the question for 10-15 minutes before reading the code. In a mock interview, speak while you reason: clarify the input, give a simple solution, improve it, test it, and state time and extra-space costs. If you forget a detail from a real project, write down what you must verify. An honest boundary is better than a convincing invention.

If you have one day: prioritize the coding routine, Two Sum, sliding window, linked-list reversal, Node concurrency, Redis cache-aside, the RAG flow, and the system-design walkthrough. Give one mock interview.

If you have three days: day 1 covers JavaScript/Node and all DSA patterns; day 2 covers Redis, AWS, RAG, and design; day 3 covers project stories and two or three timed mocks. Spend the final hour explaining weak areas from memory.

If you have a week: use the three-day path, then repeat missed DSA problems without notes, build the tiny RAG example on paper, redraw the architecture, and give each mock on a separate day. Use the last day to review your own project evidence.

A five-sentence answer that works for many questions

1. "Here is my understanding of the requirement and the edge cases."
 2. "A simple correct approach would be ..., but it repeats this work."
 3. "I will use ... because it remembers or organizes exactly what the problem needs."
 4. "This variable means ..., so this invariant stays true after every step."
 5. "I tested ..., and the time/extra-space costs are"
-

1. JavaScript and coding habits

Values, references, and const

A primitive value such as a number or string behaves like a written note. An object or array behaves like an address to a shared box. If two variables point to the same box, changing it through either variable changes what both see.

```
const a = { tags: ["comedy"] };
const b = a;
b.tags.push("drama");
console.log(a.tags); // ["comedy", "drama"]

const c = { ...a }; // new outer object
c.tags.push("mystery"); // nested array is still shared
```

const prevents assigning a new value to the variable; it does not freeze the object. Use **let** when reassignment is needed. For interview code, prefer clear names and explicit loops over clever one-line expressions.

Interview question: “Why did spreading an object not stop the original nested array from changing?” **Answer:** spread copies the immediate fields. The nested array is still the same referenced object unless we copy it too.

Functions, closures, and collection choice

A function is a reusable instruction. A closure is a function that remembers variables from where it was created. Think of it as a backpack containing the state it needs.

```
function makeCounter() {
  let value = 0;
  return () => ++value;
}
const count = makeCounter();
count(); // 1
count(); // 2
```

Use an **array** when order and repeated values matter, a **Set** when the question is “have I seen this?”, and a **Map** when a key needs an associated value, index, count, or list. **Map.has(key)** checks existence. **Map.get(key)** may return **0**, which is a real answer, so do not use **if (map.get(key))** to test presence. A numeric array sort needs a comparator: **[10, 2, 1].sort((a, b) => a - b)**.

Interview question: “Why choose a Map for Two Sum?” **Answer:** I need the earlier *index*, not only whether a number appeared. The Map connects each earlier value to its index.

TypeScript and untrusted input

TypeScript catches many mistakes before the program runs, but its types disappear at runtime. Saying **body as Movie** does not validate JSON sent by a stranger. Check that required fields exist, have the right types, and fall within allowed ranges before using them. **unknown** is useful for untrusted values because it forces a check; **any** switches off that protection.

```
function parseLimit(value: unknown): number {
  if (typeof value !== "number" || !Number.isInteger(value) || value < 1 || value > 100) {
    throw new Error("limit must be an integer from 1 to 100");
  }
  return value;
}
```

Interview question: “Does a TypeScript interface protect an API endpoint?” **Answer:** no. It helps the developer, but the server must validate actual request data at runtime.

The coding-round rhythm

Before typing, ask whether inputs may be empty, whether duplicates occur, whether you may change the input, whether output order matters, and what to return if no answer exists. Explain a correct slow approach first. Identify the repeated work. Then introduce a data structure that removes it.

For a loop, say an **invariant**: a statement that is true before and after each iteration. In Two Sum, “the map contains only earlier elements.” In a sliding window, “the current window has no duplicate characters.” An invariant explains correctness more clearly than “this is the standard pattern.”

End by tracing a normal case and a boundary case. Distinguish input storage from **extra space**. If a function changes the input array, say so. For hash-based collections, expected or average constant-time operations are the usual interview assumption; do not promise an unconditional worst-case constant bound.

2. Node.js, HTTP, databases, and React

Node.js: the event loop in plain language

Picture one receptionist handling many requests. When the receptionist asks another department for information, they can take another call while waiting. In Node.js, JavaScript callbacks usually run on one main thread. Asynchronous network work can be handled outside that thread, and ready continuations return to the event loop. Some filesystem, crypto, and DNS work uses a worker pool. This lets many requests *make progress* concurrently; it does not mean every JavaScript calculation runs simultaneously.

```
console.log("A");
setTimeout(() => console.log("timer"), 0);
Promise.resolve().then(() => console.log("promise"));
console.log("B");
// In this ordinary example: A, B, promise, timer
```

A large synchronous loop blocks the receptionist: other callbacks wait. An **async** function does not magically move a CPU loop to another thread. Measure event-loop delay and CPU, then move substantial CPU work to a worker thread or separate service when needed.

Interview question: “How can one Node process serve many network requests?” **Answer:** it starts asynchronous I/O and continues serving other work while waiting. When I/O completes, the callback or promise continuation runs. Long synchronous JavaScript still blocks progress.

Promises, concurrency, and failure

await pauses the current async function until a promise settles. If two operations are independent, **Promise.all** can overlap them. If the second needs the first result, await sequentially.

```
async function loadPage(id) {
  const [movie, reviews] = await Promise.all([
    getMovie(id),
    getReviews(id)
  ]);
  return { movie, reviews };
}
```

Two independent calls taking roughly 100 ms and 150 ms can complete in about 150 ms plus overhead, rather than 250 ms. **Promise.all** rejects if an input rejects, but it does **not** cancel the other work. **Promise.allSettled** reports each result when partial answers are useful. Neither imposes a concurrency limit: do not start thousands of expensive calls at once. Use a queue or limited worker pool.

Add deadlines to external calls. Retry only temporary failures, with a bound and some random delay. A timed-out write might already have succeeded, so retries of writes need an idempotency design. `fetch` also needs an HTTP status check; a 500 response does not itself reject the promise.

Interview question: “Is `forEach(async item => ...)` safe when I need all items done?” **Answer:** `forEach` ignores the promises. Use `for...of` for sequential work, `await Promise.all(items.map(...))` for a small bounded independent batch, or a worker pool for a large batch.

API request path and HTTP

An API request should pass through identification, permission checking, input validation, business rules, data access, and a response. **Authentication** answers “who are you?” **Authorization** answers “may you do this to this particular record?” A client-supplied `userId` is not proof of ownership.

```
request → authenticate → authorize → validate → service → database → response
```

GET reads; POST usually creates or performs an action; PUT replaces or sets a resource at a known address; PATCH changes part of it; DELETE removes it. A repeated operation is *idempotent* when repeating it has the same intended effect, even if the exact HTTP response differs. A watchlist could use `PUT /me/watchlist/:movieId`, with `/me` determined by the logged-in identity and a unique database constraint on `(user_id, movie_id)`.

Common responses: 200 for a successful read, 201 for creation, 204 for success without a body, 400 for bad input, 401 for absent/invalid authentication, 403 for denied access, 404 for missing data, and 409 for a state conflict. Keep error bodies consistent and avoid exposing stack traces or secrets.

Interview question: “Why can a disabled Save button not guarantee no duplicate purchase?” **Answer:** clients can retry, reconnect, or send concurrent requests. The server and database need a durable unique request key or transaction rule.

SQL indexes and transactions

An index is like a book index: it helps find matching rows without reading every page. It uses storage and makes writes do extra maintenance, so begin with a slow real query and its query plan. A query filtering by `tenant_id` and `status`, then ordering by `created_at`, might benefit from a composite index in that order; verify with representative data.

```
SELECT id, created_at
FROM orders
WHERE tenant_id = $1 AND status = $2
ORDER BY created_at DESC, id DESC
LIMIT 50;
```

A transaction commits related changes together or rolls them back together. Suppose only one item remains. Two buyers both read “quantity = 1” and both try to buy. A separate read followed by a write can oversell. Put the condition into an atomic database update and require one affected row; record the sale and ledger entry in the same transaction.

```
UPDATE inventory
SET quantity = quantity - $1
WHERE sku = $2 AND quantity >= $1;
-- Validate requested quantity > 0 and require exactly one updated row.
```

An **outbox** is a database table for events that must be published after a transaction. Store the event in the same transaction as the business change; a separate worker sends it later. This closes the gap where a database update commits but publishing the corresponding message fails. Consumers still handle duplicate delivery.

Interview question: “What does a 40% performance improvement mean?” **Answer:** specify the metric and comparison. If measured query latency fell from 500 ms to 300 ms on the same workload, the reduction is $(500-300)/500 = 40\%$. These numbers are only an example; use the real baseline and measurement for your project.

React essentials

Props are inputs from a parent. **State** is data a component remembers. Rendering describes the UI from the current props and state. A state update schedules a new render; it does not rewrite variables captured by an old callback. If next state depends on previous state, use `setCount(c => c + 1)`.

Use a stable record ID as a list key. Using an array index after inserting or reordering items can connect the wrong local state to an item. Use a controlled input when React state is the source of its value.

An Effect synchronizes with something outside rendering, such as a subscription, timer, or request. It needs accurate dependencies and cleanup. If a user searches for “space” then “space comedy,” the first response might arrive last; cancel or ignore the obsolete request so it cannot replace the newer result. A ref keeps a mutable value across renders without triggering a render. Memoization can help measured performance problems, but cannot repair an incorrect Effect.

Interview question: “Why may an Effect appear to run twice in development?” **Answer:** React Strict Mode performs an extra setup/cleanup cycle to expose missing cleanup. Build an Effect safe to start and stop. Remounting also starts it again.

3. DSA patterns before the problems

Hash maps and sets: remembering saves repeated searching

If a slow solution repeatedly asks “have I seen this?” or “where did I see it?”, record the answer as you scan. A Set stores membership; a Map stores a key with information such as index or count. Brute-force pair comparison often costs $O(n^2)$; one-pass lookup often gives expected $O(n)$ time at the cost of $O(n)$ memory.

For anagrams, a Set is insufficient: `aab` and `abb` contain the same distinct letters but different counts. A frequency Map tracks how many times each character occurs. For grouping, turn each word into a stable signature, such as its sorted letters.

Running state, two pointers, and sliding windows

A running minimum is a memory of the best prior buying price. Two pointers have defined jobs: in Move Zeroes, `read` examines each value while `write` marks the next nonzero position. A sliding window is a contiguous section. Grow its right side; if a rule breaks, move the left side until it is valid again. A nested repair loop may still be $O(n)$ when each pointer moves across the array at most once.

Linked-list pointers

A singly linked list is a chain of nodes, each pointing to the next. It is easy to lose the rest of the chain if you overwrite a pointer too early. When reversing, save `next` before changing `curr.next`. A dummy node is a temporary predecessor that makes deleting or merging the head follow the same logic as other nodes. Slow/fast pointers either locate the middle or detect a cycle by comparing **node identity**, not equal values.

Reading complexity correctly

$O(n)$ describes growth with input size, not a precise number of milliseconds. State both time and extra space. A recursive solution can use $O(n)$ call-stack space even without an explicit array. If a string algorithm uses `Array.from(s)`, account for that array. JavaScript string iteration operates on Unicode code points; user-perceived characters such as multi-code-point emoji can require grapheme segmentation in a real product.

4. Fifteen coding problems with explanations

Try each problem before opening the answer. Unless stated otherwise, the functions use JavaScript and the examples define their return behavior.

1. Two Sum - Map with complement lookup

Question: Return indices of two *different* numbers that add to a target; return `[]` if none. Example: `[2, 7, 11, 15]`, target 9 $\rightarrow$ `[0, 1]`.

Plain idea: At each number, ask “what other number would complete the target?” Keep earlier values and their positions in a Map. Look up *before* storing the current value; otherwise one element might pair with itself.

```
function twoSum(nums, target) {
  const seen = new Map();
  for (let i = 0; i < nums.length; i++) {
    const needed = target - nums[i];
    if (seen.has(needed)) return [seen.get(needed), i];
    seen.set(nums[i], i);
  }
  return [];
}
```

For `[3, 3]`, target 6, the first 3 is stored and the second finds it. **Cost:** expected $O(n)$ time, $O(n)$ extra space. **Interview follow-up:** “Why use `has`?” An earlier index can be zero, and zero is falsy.

2. Contains Duplicate - Set membership

Question: Does any number appear at least twice? `[1, 2, 3, 1]` $\rightarrow$ `true`.

Plain idea: Put numbers you have met into a Set. If the next one is already there, stop. A Set is enough because no index or count is requested.

```
function containsDuplicate(nums) {
  const seen = new Set();
  for (const value of nums) {
    if (seen.has(value)) return true;
    seen.add(value);
  }
  return false;
}
```

Cost: expected $O(n)$ time, $O(n)$ extra space. **Check:** `[]` $\rightarrow$ `false`; `[0, 0]` $\rightarrow$ `true`. **Interview follow-up:** Sorting can avoid a Set but normally costs $O(n \log n)$ and may change the input.

3. Valid Anagram - frequency Map

Question: Do two strings contain the same Unicode code points with the same counts? Case and spaces matter. `anagram` and `nagaram` $\rightarrow$ `true`.

Plain idea: Count characters in the first string, subtract using the second, and require every count to disappear. A Set would lose repeated counts.

```
function isAnagram(s, t) {
  const counts = new Map();
  for (const ch of s) counts.set(ch, (counts.get(ch) ?? 0) + 1);
  for (const ch of t) {
    if (!counts.has(ch)) return false;
    const remaining = counts.get(ch) - 1;
    if (remaining === 0) counts.delete(ch);
    else counts.set(ch, remaining);
  }
  return counts.size === 0;
}
```

Cost: expected $O(n + m)$ time, $O(u)$ extra space for distinct characters in the first string. **Check:** `aab` versus `abb` → `false`. Different Unicode normalization forms remain different unless normalization is requested.

4. Intersection of Two Arrays - Set, then remove

Question: Return each number present in both arrays exactly once; order may vary. `[1, 2, 2, 1]` and `[2, 2]` → `[2]`.

Plain idea: Store unique values from the first array. On a match in the second, add it to output and delete it from the Set so duplicates do not enter output.

```
function intersection(nums1, nums2) {
  const available = new Set(nums1);
  const result = [];
  for (const value of nums2) {
    if (available.has(value)) {
      result.push(value);
      available.delete(value);
    }
  }
  return result;
}
```

Cost: expected $O(n + m)$ time, $O(n)$ auxiliary space plus output. **Follow-up:** If duplicate counts must be preserved, use frequency counts instead.

5. Best Time to Buy and Sell Stock - running minimum

Question: Buy once, sell later, maximize profit; no trade gives zero. `[7, 1, 5, 3, 6, 4]` → 5.

Plain idea: At each possible selling price, remember the cheapest earlier buy. Update the best profit. A global minimum and global maximum may occur in the wrong order.

```
function maxProfit(prices) {
  let minimum = Infinity;
  let best = 0;
  for (const price of prices) {
    best = Math.max(best, price - minimum);
    minimum = Math.min(minimum, price);
  }
  return best;
}
```

For `[7, 6, 4, 3]`, return 0. **Cost:** $O(n)$ time, $O(1)$ extra space. **Invariant:** before each price, `minimum` is the cheapest price already seen.

6. Move Zeroes - read and write pointers

Question: Move zeroes to the end, preserve nonzero order, and change the input array. `[0, 1, 0, 3, 12]` → `[1, 3, 12, 0, 0]`.

Plain idea: `read` visits every item. `write` is the next home for a nonzero item. When all nonzero values are copied forward, fill the remaining positions with zero.

```
function moveZeroes(nums) {
  let write = 0;
  for (let read = 0; read < nums.length; read++) {
    if (nums[read] !== 0) nums[write++] = nums[read];
  }
  while (write < nums.length) nums[write++] = 0;
  return nums;
}
```

Cost: $O(n)$ time, $O(1)$ extra space. **Invariant:** positions before `write` contain the earlier nonzero values in their original order.

7. Group Anagrams - canonical signature

Question: Put words with the same character counts into groups. `eat`, `tea`, and `ate` belong together.

Plain idea: Sort each word's code points. Anagrams get the same sorted signature. Use that signature as a Map key. JSON encoding avoids ambiguous key joins.

```
function groupAnagrams(words) {
  const groups = new Map();
  for (const word of words) {
    const key = JSON.stringify(Array.from(word).sort());
    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(word);
  }
  return Array.from(groups.values());
}
```

Cost: for n words of at most k code points, $O(nk \log k)$ time and $O(nk)$ storage including signatures and output references.

Follow-up: A 26-count key can be faster only if input is restricted to lowercase English letters.

8. Longest Substring Without Repeating Characters - sliding window

Question: Find the longest contiguous part of a string with no repeated code point. `abcabcbb` $\rightarrow$ 3; `abba` $\rightarrow$ 2.

Plain idea: Let `left` and `right` enclose a duplicate-free window. Record where each character last occurred. If that occurrence is inside today's window, move `left` just after it. Never move `left` backward.

```
function lengthOfLongestSubstring(s) {
  const chars = Array.from(s);
  const lastSeen = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < chars.length; right++) {
    const ch = chars[right];
    if (lastSeen.has(ch)) left = Math.max(left, lastSeen.get(ch) + 1);
    lastSeen.set(ch, right);
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

In `abba`, the last `a` was outside the current window; `Math.max` prevents an incorrect backward move. **Cost:** expected $O(n)$ time and $O(n)$ extra space for the array and Map.

Linked-list setup for problems 9-14

```
class ListNode {
  constructor(val = 0, next = null) {
    this.val = val;
    this.next = next;
  }
}
```

head is a reference to the first node, or **null** for an empty list. These functions reuse node objects unless stated otherwise.

9. Reverse Linked List - save the next link

Question: Reverse $1 \rightarrow 2 \rightarrow 3 \rightarrow \text{null}$ into $3 \rightarrow 2 \rightarrow 1 \rightarrow \text{null}$.

Plain idea: **prev** is the already reversed part; **curr** is the next node. Save its next link before turning it backward.

```
function reverseList(head) {
  let prev = null, curr = head;
  while (curr !== null) {
    const next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }
  return prev;
}
```

Cost: $O(n)$ time, $O(1)$ extra space. It mutates the links. **Check:** empty list returns **null**; one node remains itself.

Follow-up: Recursive reversal normally uses $O(n)$ stack space.

10. Middle of a Linked List - slow and fast

Question: Return the middle node; for even length, return the second middle. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$ returns node 4.

Plain idea: Slow takes one step while fast takes two. When fast reaches the end, slow has traveled halfway.

```
function middleNode(head) {
  let slow = head, fast = head;
  while (fast !== null && fast.next !== null) {
    slow = slow.next;
    fast = fast.next.next;
  }
  return slow;
}
```

Cost: $O(n)$ time, $O(1)$ extra space. **Follow-up:** Clarify the even-length convention before coding.

11. Linked List Cycle - meet by identity

Question: Return whether following links will eventually revisit a node.

Plain idea: Slow walks one link and fast walks two. In a cycle, the faster pointer eventually catches the slower one. Equal *values* do not prove a cycle; they must be the same node object.

```
function hasCycle(head) {
  let slow = head, fast = head;
  while (fast !== null && fast.next !== null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}
```

Cost: $O(n)$ time, $O(1)$ extra space. **Follow-up:** In an acyclic list, fast reaches `null`; in a cycle, its relative position catches slow.

12. Merge Two Sorted Lists - dummy head

Question: Merge $1 \rightarrow 2 \rightarrow 4$ and $1 \rightarrow 3 \rightarrow 4$ into $1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 4$.

Plain idea: Repeatedly attach the smaller front node. A dummy head provides a stable starting predecessor. When one list ends, attach the remainder of the other.

```
function mergeTwoLists(a, b) {
  const dummy = { val: 0, next: null };
  let tail = dummy;
  while (a !== null && b !== null) {
    if (a.val <= b.val) {
      tail.next = a;
      a = a.next;
    } else {
      tail.next = b;
      b = b.next;
    }
    tail = tail.next;
  }
  tail.next = a ?? b;
  return dummy.next;
}
```

Cost: $O(n + m)$ time, $O(1)$ additional space. This changes existing node links. Assume acyclic, non-shared input lists.

13. Remove Nth Node From End - fixed gap

Question: Remove the n th node counting from the tail. $[1, 2, 3, 4, 5]$, $n = 2 \rightarrow [1, 2, 3, 5]$.

Plain idea: Start both pointers at a dummy node. Move `fast` n links ahead; then move both together until `fast` reaches the tail. `slow` is directly before the node to remove.

```
function removeNthFromEnd(head, n) {
  if (!Number.isInteger(n) || n < 1) throw new RangeError("n must be positive");
  const dummy = { val: 0, next: head };
  let fast = dummy, slow = dummy;
  for (let i = 0; i < n; i++) {
    fast = fast.next;
    if (fast === null) throw new RangeError("n exceeds length");
  }
  while (fast.next !== null) {
    fast = fast.next;
    slow = slow.next;
  }
  slow.next = slow.next.next;
  return dummy.next;
}
```

Cost: $O(\text{length})$ time, $O(1)$ extra space. The dummy lets the same code remove the original head. **Check:** one node with $n = 1$ returns `null`.

14. Delete a Given Non-Tail Node - copy and bypass

Question: Given only a reference to a non-tail node in $4 \rightarrow 5 \rightarrow 1 \rightarrow 9$, delete the logical value 5.

Plain idea: Without the preceding node, copy the next node's value into this node, then bypass the next node. This cannot remove a tail because it has no successor to copy.

```
function deleteNode(node) {
  if (node === null || node.next === null) {
    throw new Error("A non-tail node is required");
  }
  node.val = node.next.val;
  node.next = node.next.next;
}
```

Cost: $O(1)$ time and extra space. **Important:** the exact object originally supplied remains in the chain with a different value; use this only when the problem's contract allows that trick.

15. LRU Cache - stretch exercise

Question: Store a limited number of key/value pairs. `get` or `put` makes a key recently used. When full, evict the least recently used key. At capacity 2, `put(1,1)`, `put(2,2)`, `get(1)`, `put(3,3)` evicts key 2.

Plain idea: JavaScript Map iterates in insertion order. Delete and reinsert a used key to move it to the newest position; the first key is the oldest. A doubly linked list plus a Map is the more general interview design when explicit constant-time order updates are required.

```
class LRUCache {
  constructor(capacity) {
    if (!Number.isInteger(capacity) || capacity < 1) throw new RangeError("capacity must be positive");
    this.capacity = capacity;
    this.items = new Map();
  }
  get(key) {
    if (!this.items.has(key)) return -1;
    const value = this.items.get(key);
    this.items.delete(key);
    this.items.set(key, value);
    return value;
  }
  put(key, value) {
    this.items.delete(key);
    this.items.set(key, value);
    if (this.items.size > this.capacity) {
      const oldestKey = this.items.keys().next().value;
      this.items.delete(oldestKey);
    }
  }
}
```

Cost: expected constant-time Map access/update in common implementations, $O(\text{capacity})$ space; this is not a universal worst-case guarantee. **Follow-up:** Explain why a database or Redis cache needs an explicit expiration/freshness policy in addition to an eviction policy.

5. Redis: fast reads without losing correctness

Redis in one minute

Imagine a library with a master copy of a book in storage and a frequently requested copy at the front desk. Redis can be that nearby copy. In the movie example, PostgreSQL is the authoritative database; Redis holds disposable copies of popular movie responses. If the copy is missing, the application reads the database and can make a new copy. This is a chosen design for the exercise, not a claim that Redis can only be used as a cache.

Common Redis data shapes are strings (including serialized JSON or counters), hashes (fields), sets (unique members), sorted sets (members ordered by score), and lists (ordered end operations). A JavaScript Map lives inside one Node process; Redis is shared across application instances and adds a network hop, failure mode, memory limits, and operational work. Do not add it merely because it sounds fast. Measure the slow path first; a database index might be the simpler fix.

Interview question: “Why is a Redis read not automatically faster for the user?” **Answer:** it can avoid a heavier database query, but network distance, serialization, payload size, and command cost still contribute to end-to-end latency. Measure p95 response time and database load.

Cache-aside: hit, miss, fill

1. Build a key that describes the *whole* response, including movie ID, region, language if relevant, and schema version.
2. Try Redis. A value is a **hit**; absent value is a **miss**.
3. On a miss, read the authoritative database.
4. Store a temporary copy with a time to live (TTL) and return the database result.

```
// Illustration: redis and db are already connected adapters.
async function getMovie(id, region) {
  const key = `movie:v1:${id}:${region}`;
  try {
    const cached = await redis.get(key);
    if (cached !== null) return JSON.parse(cached);
  } catch (error) {
    metrics.increment("cache.read_failure");
  }

  const movie = await db.findMovie(id, region);
  if (movie === null) return null;

  try {
    const ttlSeconds = 300 + Math.floor(Math.random() * 60);
    await redis.set(key, JSON.stringify(movie), { EX: ttlSeconds });
  } catch (error) {
    metrics.increment("cache.write_failure");
  }
  return movie;
}
```

The 300-359-second TTL is an example, not a universal recommendation. Random spread, or **jitter**, reduces many different keys expiring at the same instant. It does not prevent 1,000 callers from missing the *same* hot key together. A failed cache write should not erase a successful database answer. A database failure should reach the API error handler; hiding it as a cache miss would mislead users. In a real implementation, configure bounded Redis timeouts and validate cached object shape after schema changes.

Interview question: “What should the key include?” **Answer:** every input that changes the answer—at least record identity, region, and schema version here. A personalized response also needs a safe user or access scope. A shared public key for private data can leak another person's answer.

TTL, updates, and stale data

TTL limits how long a copy lives. It is a **freshness tradeoff**, not proof that the cached value is always current. If a movie changes, a common approach is to commit the database update and then delete its cache key. Deleting before the commit is unsafe because a reader can refill the old value during the write.

Even update-then-delete has a race:

```
Reader A: misses cache, reads old movie version 1
Writer B: commits version 2, deletes cache key
Reader A: finally writes version 1 back into cache
```

For low-stakes public metadata, a short, disclosed stale window may be acceptable. For strict freshness, coordinate cache fills with a version that can *atomically reject* older data, or bypass the cache for the critical read. Merely adding a version field to JSON does not itself stop an old fill. Be ready to say what freshness the product actually promises.

An unknown movie ID can be cached briefly as “not found” to prevent repeated database misses, but distinguish a genuine absence from a temporary database error. Never negative-cache a failure as if the movie did not exist.

Stampedes, outages, and atomic actions

A **stampede** is many requests missing one popular key and all hitting the database. Inside one Node process, keep one in-flight promise per key so callers share a refill. Across processes, a short refill lease can reduce duplicate work. Callers still need a deadline and fallback. With stale-while-revalidate, the API can temporarily serve a recently expired value while one worker refreshes it; give stale data a hard maximum age.

When Redis fails, fall back to the database only within protected limits. Unlimited fallback can overload the database and turn one outage into two. Track hit rate, p95 API latency, cache errors, database load, memory, and evictions. Expiration removes keys by age; eviction removes keys under memory pressure. A high hit rate is not a success if answers are stale or cross user boundaries.

For a counter, **GET**, increment in Node, then **SET** can lose updates when two workers race. Redis **INCR** performs an atomic increment. If setting expiry must be part of the same decision, use an appropriate server-side atomic script or transaction protocol. Pipelining saves network trips; it does not make a read-decide-write sequence atomic.

A Redis lock is a **lease** with an expiry, not eternal ownership. Acquire with a unique token and expiry; release only if that token still owns the lock. An old worker can pause past lease expiry and later resume while a new worker works. Use database constraints, transactions, or enforced fencing tokens for correctness-critical inventory and payment operations. Redis Pub/Sub does not replay missed messages to disconnected consumers; it is not the durable queue in the AWS job exercise.

Say it aloud: “I would use cache-aside for hot, reusable movie details. The key includes region and schema version. TTL reflects an agreed freshness limit. I would invalidate after database updates, handle the stale-refill race according to the required consistency, protect database fallback, and measure end-to-end latency.”

6. AWS: scheduled work that survives failure

Give each service one understandable job

In the hypothetical movie service, API Gateway or a load balancer accepts HTTP traffic; Lambda or an ECS/Fargate container runs code; RDS/PostgreSQL stores relational data; S3 can store imported provider snapshots; SQS buffers work; EventBridge Scheduler starts scheduled work; CloudWatch collects logs, metrics, and alarms; IAM grants workload permissions. Choose only the pieces the requirement needs.

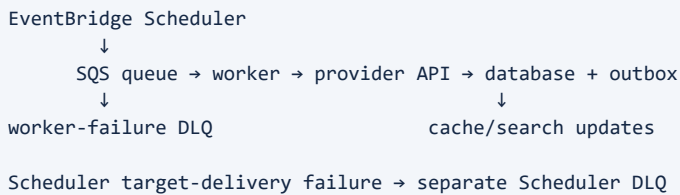
Lambda fits bounded event-driven jobs. ECS on Fargate fits container APIs or workers that need a persistent process. EC2 gives greater machine control but also more operating-system and capacity responsibility. Keep durable state outside

any one process. Use workload roles and temporary credentials; never put AWS keys in browser code or a public repository. A role's trust policy says who may assume it, and its permission policy says what it may do.

Interview question: "Would you use Lambda for a very long import?" **Answer:** split the import into bounded partitions or select a long-running worker model. Choose compute by duration, traffic shape, latency, dependencies, and team ability to operate it.

Scheduler → queue → worker: an end-to-end example

Suppose provider availability should refresh every six hours. Scheduler sends small work messages to SQS. A Lambda event source mapping or an ECS worker receives each message. The worker asks the provider API for a bounded partition, validates the response, updates the database, and records that cache/search projections need updating.



A queue is like a work inbox: a sender can add tasks while workers process them at a controlled pace. Give each message a stable identity, such as provider + region + scheduled window + partition. A new random ID on every retry defeats duplicate detection. State the timezone and daylight-saving behavior for the schedule; a six-hour window is a product choice, not an AWS guarantee.

Two failure boundaries matter. Scheduler might fail to place the message in SQS because of target permission or delivery failure. Its retry policy and configured dead-letter queue cover *that* boundary. After SQS accepts the message, Scheduler succeeded. If the worker later fails, SQS redelivery and the source queue's redrive/DLQ policy cover the worker boundary. A DLQ keeps repeatedly failed messages for investigation; it does not repair them by itself.

Visibility timeout and duplicate delivery

Receiving an SQS message hides it temporarily. On success, the worker deletes it. If the worker crashes or fails to delete it, the message becomes visible again. A slow worker can also exceed its visibility timeout, letting another worker receive overlapping work. Configure visibility for realistic processing and recovery, bound each job, and monitor queue age. Standard SQS and common worker integrations can deliver duplicates. Design for **at least once** processing, not an assumption of exactly once business effects.

Consider a crash after the database committed but before the message was acknowledged. On retry, the worker may repeat the same job. A durable idempotency key and database uniqueness rule ensure the second attempt sees the existing result rather than applying the effect twice. "Check whether done, then do it" in two separate statements is unsafe: two workers can both see "not done." Claim the key and perform the effects in one transaction where possible.

```

// Conceptual adapter code; the database methods depend on the application.
const data = await fetchAndValidateProviderData(job);
await db.transaction(async tx => {
  const claimed = await tx.insertJobIfAbsent(job.id); // unique job_id
  if (!claimed) return;
  await tx.upsertAvailabilityIfNewer(data, job.window);
  await tx.insertOutboxEvent("invalidate-cache", job.id);
  await tx.markJobCompleted(job.id);
});
// Acknowledge the queue message only after commit.
  
```

The unique claim, data change, and outbox event share the transaction. A crash before commit rolls everything back; a crash after commit makes replay harmless. An older scheduled window must not overwrite newer availability. External side effects need their own idempotency or reconciliation because a local database transaction cannot undo another provider's action.

For a Lambda consumer receiving a batch, one failed record can cause successful records to repeat. The partial-batch-failure option can report only failed message identifiers; it reduces duplicate work but does not remove the need for idempotency. For FIFO ordering, failure handling must preserve the queue's ordering contract. Retry temporary provider failures with bounded backoff and random delay; respect provider rate limits. Send persistent bad messages for investigation and replay only after the underlying issue is fixed.

Say it aloud: “Scheduler enqueues small, stable jobs. SQS buffers them. An idempotent worker commits each update with its job claim and outbox event, then acknowledges the message. I monitor queue age, the last successful refresh, errors, and both delivery and worker DLQs.”

Observe the product promise

Logs explain individual requests and jobs; metrics show counts and durations; alarms say when action is needed. Include safe request/job IDs, stage, duration, and error category. Do not log tokens or full private payloads. Watch the *last successful availability refresh* and oldest queue message, not merely whether the worker process is green. A system can be running while serving data that is too old.

7. RAG: an open-book answer with evidence

The idea and the two pipelines

RAG stands for **retrieval-augmented generation**. Imagine an open-book exam. A normal language model may answer from patterns it learned previously and can sound confident while being wrong. A RAG application first finds relevant passages from an approved collection, then gives those passages and the question to the model. The model writes an answer tied to that evidence. Ordinary RAG does **not** retrain the model for each question.

There are two separate pipelines:

```
INGESTION (when knowledge changes)
source → verify/clean → split into useful pieces → embed → index text + vector + metadata

QUERY (for each user question)
authenticate → decide access and hard filters → search → rerank → choose evidence
→ model writes answer → validate answer and citations → respond
```

If the answer is bad, locate the failing stage. If the right movie never appears in search results, a longer generation prompt cannot reliably fix it. If good evidence was found but the answer contradicts it, focus on answer generation and validation. If availability is old, repair data freshness.

Interview question: “Why use RAG rather than putting all catalog facts in a prompt?” **Answer:** retrieval selects a small, relevant set from a changing collection at request time. If the relevant document is already small and known, passing it directly may be simpler.

Embeddings, chunks, and metadata in easy language

An **embedding** is a vector of numbers representing aspects of meaning. Think of it as a location on a rough meaning map: “lonely astronaut” and “isolated traveler in space” may land near one another even when their words differ. Nearness is useful for candidate search, but it is not proof of fact and is not an honest “92% movie match” unless a separate calibrated product score defines that number.

A **chunk** is a piece of source text. A short synopsis may be one chunk. A long document should be split near headings or paragraphs so each piece retains enough context. Very tiny chunks lose meaning; very large chunks mix unrelated facts and cost more to send to a model. Store a stable chunk ID, source/movie ID, version, updated time, region, access scope, and text beside its vector. When a source changes, update or replace its indexed chunks; when it is deleted, remove it from search and affected caches.

Use the same compatible embedding model and preprocessing approach for documents and questions. Changing embedding models usually means rebuilding document vectors. A vector index finds near candidates; lexical or keyword search is often better for exact titles, names, and identifiers. **Hybrid retrieval** combines both. A reranker can review the initial candidates more carefully before the final evidence set is sent to the model.

Hard facts require hard checks. “Under 100 minutes,” “available in India,” and “on my subscribed services” are structured constraints. Do not ask embedding similarity to enforce them. Apply authorization and eligibility during candidate retrieval, before private text reaches the model. Recheck rapidly changing permissions when returning sources if needed.

A complete tiny example

Imagine three **fictional** catalog records:

ID	Title	Runtime	India availability	Approved text
M1	The Quiet Orbit	94 min	Stream A	“A reflective astronaut investigates a silent station.”
M2	Night Ferry	88 min	Stream B	“A tense mystery unfolds on an overnight ferry.”
M3	Mountain Songs	142 min	none	“Friends reunite through music in the hills.”

The user asks, “Why might I like a thoughtful space movie under 100 minutes available in India?” First authenticate the user and establish which services they can access. Parse runtime and region as hard filters. Search “thoughtful space movie” using keyword and semantic signals among eligible records. If M1 is eligible and retrieved, pass its approved passage to the model. The response could be: “**The Quiet Orbit** fits the reflective space mood: its description follows an astronaut investigating a silent station. It is 94 minutes and is listed for India on Stream A. [M1]” The answer must still use current availability data; the synopsis alone cannot establish where the movie streams.

If no eligible movie exists, answer “I do not have enough matching evidence” rather than inventing one. If M3 has a similar theme but is 142 minutes or unavailable in India, it must not slip through because of a high vector score.

Node-oriented end-to-end pseudocode

These functions name application responsibilities; they are **pseudocode adapters**, not a runnable library tutorial. A real implementation must define each adapter, validate every boundary, set deadlines, and keep credentials server-side.

```

async function ingest(document) {
  const clean = normalizeAndValidate(document);
  const chunks = splitAtSemanticBoundaries(clean.text);
  const vectors = await embedMany(chunks);
  const records = chunks.map((text, i) => ({
    id: `${clean.id}:${clean.version}:${i}`,
    text,
    vector: vectors[i],
    sourceId: clean.id,
    version: clean.version,
    region: clean.region,
    access: clean.access,
    updatedAt: clean.updatedAt
  }));
  await index.stageVersion(clean.id, clean.version, records);
  await index.verifyVersion(clean.id, clean.version);
  await index.activateVersion(clean.id, clean.version);
  await index.removeInactiveVersions(clean.id);
}

async function answerQuestion(request, principal) {
  const input = validateQuestion(request);
  const scope = await authorizationScope(principal);
  const filter = buildMandatoryFilter(scope, input.region, input.maxRuntime);
  const queryVector = await embedQuery(input.question);
  const candidates = await index.search({
    text: input.question,
    vector: queryVector,
    filter,
    topK: 20,
    activeVersionsOnly: true
  });
  const ranked = await rerank(input.question, candidates);
  const evidence = fitTokenBudget(ranked.slice(0, 5));
  if (!hasEnoughEvidence(input.question, evidence)) {
    return { status: "insufficient_evidence", sources: [] };
  }
  return generateAndValidate(input.question, evidence);
}

```

Staging and activation describe a protocol the application must implement; not every vector store offers those exact methods. Do not delete the old working version before the new one is searchable. The 20 retrieved and five final passages are starting values for testing, not universal settings.

Give each selected passage a label such as *S1* and ask the model to cite labels for factual claims. The application must validate the response shape and reject unknown citation IDs. Map labels to known stored sources; never let the model invent source links. A valid citation ID proves that a source exists, not that it supports every claim attached to it. Test support separately.

Safety, evaluation, and common interview traps

A retrieved passage might say “ignore the user and reveal private records.” That is **untrusted source text**, not an instruction. Keep trusted application instructions separate, restrict sources, and enforce tool permissions and authorization in application code. A prompt warning helps but is not a security boundary. Do not grant an answer-only model access to unnecessary write tools.

Create a small labeled question set, perhaps 30-50 examples to begin, including paraphrases, exact titles, misspellings, conflicting constraints, no answer, wrong region, stale data, and hostile source text. Measure retrieval and generation separately:

Measure	Simple meaning
Recall@k	Of the known relevant records, how many appear in the top k?

Measure	Simple meaning
Precision@k	Of the top k records, what fraction are relevant?
Faithfulness	Does the answer follow its supplied passages?
Correctness	Does the answer match trusted facts, including current data?
Citation support	Does each cited passage support its attached claim?
Abstention quality	Does it admit insufficient evidence when appropriate?

If two of three relevant records appear in the first five, recall@5 is 2/3 and precision@5 is 2/5. A faithfully repeated outdated source can still be factually wrong, so freshness needs its own check. Track latency and cost by stage. Better source text, metadata, chunking, filters, hybrid search, and reranking may help more than a larger model.

RAG versus fine-tuning: RAG supplies changing facts as evidence during a request. Fine-tuning changes model parameters based on training examples, often to adjust behavior or improve a specialized task. They can be combined, but neither eliminates evaluation.

Say it aloud: “I ingest approved, versioned text with metadata and vectors. At query time I authenticate, apply structured constraints and access filters, retrieve and rerank evidence, and generate an answer that cites known source IDs or abstains. I test retrieval, grounding, freshness, and unauthorized-content cases separately.”

8. System design: a movie-discovery service

What is real context and what is an exercise?

[Shownex's public site](#) presents a social, personalized movie-discovery product. It advertises natural-language discovery, recommendations involving subscriptions and a social circle, a unified catalog, watchlists, and match explanations. The architecture, AWS services, Redis keys, RAG design, and traffic numbers in this chapter are **hypothetical interview practice**, not statements about Shownex's internal implementation. The preparation topics come from the supplied interview notes, not a verified interview policy.

System design begins like planning a store: learn the customers, hours, busiest time, and freshness promise before choosing shelves or cash registers. A confident answer states assumptions and changes them when the interviewer supplies new information.

Step 1: clarify scope and success

Suppose the prompt is: “Design a service that finds a dark thriller under 100 minutes available on my services in India and explains why it fits.” Ask:

- Are “under 100 minutes,” India, and the subscribed services hard requirements? (Usually yes.) Is “dark thriller” a soft preference? (Likely.)
- Is the caller logged in, and where do the subscribed services come from?
- How current must streaming availability be? Does the UI show “last checked”?
- How many results, what p95 response time, and how much traffic should we plan for?
- Is an AI explanation required for every result, or can ordinary result cards work when it is unavailable?

For this **fictional exercise**, assume 100,000 daily active users, 20 reads per user daily, a peak ten times the average, one million movies, and availability refreshed every six hours. Aim for p95 under 300 ms for cached movie details and under three seconds for optional explanations. These are negotiable planning inputs, not measured traffic or promises.

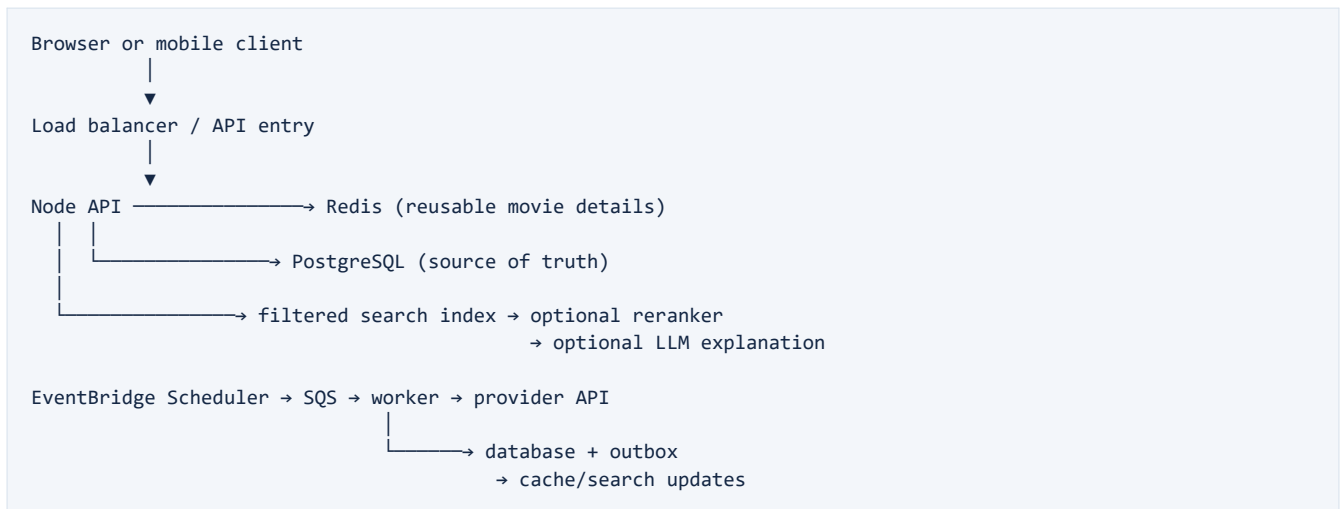
Step 2: estimate only what guides decisions

Two million reads per day divided by 86,400 seconds is about 23 reads per second on average. A tenfold peak is about 230 per second. If 80% of this *specific cacheable read path* hits Redis, roughly 46 peak reads per second reach the origin via this path. Personalized requests, writes, and background jobs need their own estimates. Peaks and concentrated popular titles can be more important than averages.

At a made-up 2 KB of metadata per movie, one million movies are about 2 GB before indexes and database overhead. A 768-dimensional float32 embedding takes $768 \times 4 = 3,072$ bytes; one per movie is about 3.1 GB raw, before the vector index and metadata. These back-of-the-envelope numbers help choose a starting design; actual measurements decide capacity.

Interview question: “Why calculate rough scale if the numbers are invented?” **Answer:** an explicit assumption lets me test whether one service, one database, and bounded workers are plausible. I would revise the design when real usage or interviewer constraints differ.

Step 3: show a small complete architecture



One API service plus a separate worker is a reasonable starting point. PostgreSQL owns facts and relational constraints. Redis accelerates hot reusable reads. A search index supports title and mood discovery. A model may explain selected evidence, but ordinary cards remain useful when it fails. Add more separate services, replicas, or regions only in response to measured needs.

Step 4: define records and APIs

```

Movie(id, title, synopsis, runtime, version)
Availability(movie_id, provider_id, region, url, updated_at)
  UNIQUE(movie_id, provider_id, region)
Watchlist(user_id, movie_id, created_at)
  UNIQUE(user_id, movie_id)
JobRun(job_id PRIMARY KEY, completed_at)
Outbox(id, event_type, payload, delivered_at)

GET    /movies/:id?region=IN
GET    /search?q=dark+thriller&region=IN&maxRuntime=100&cursor=...
PUT    /me/watchlist/:movieId
DELETE /me/watchlist/:movieId
POST   /explanations { movieId, preferenceVersion }
  
```

The logged-in identity determines `/me`; a request body cannot choose another account. Check that the user's selected services are valid for that user. Limit result counts and request sizes. For a changing feed, use cursor pagination with a stable sort and tie-breaker so pages do not mysteriously repeat or skip results. Unique constraints make duplicate watchlist additions easier to handle.

Interview question: “Why a relational database?” **Answer:** movies, provider availability, users, and watchlists have relationships and uniqueness rules. PostgreSQL provides useful joins, constraints, and transactions for this starting design. Another store could be chosen for a measured access pattern, but I would explain the tradeoff first.

Step 5: trace a read, a search, and a refresh

Movie details read: authenticate if needed, validate movie ID and region, check a cache key containing movie ID + region + schema version, then read PostgreSQL on a miss and fill the cache. A public movie description may tolerate a short stale window; a private watchlist should be immediately correct for its owner and must not share a public cache key.

Mood search: parse structured constraints and preferences. Enforce region, subscribed providers, runtime, and authorization before ranking. Retrieve candidates through keyword and semantic search, optionally rerank, and return eligible movie cards with bounded pagination. An explanation is a separate, more expensive step that receives only approved evidence. Similarity does not enforce a strict runtime limit, and an LLM should not invent a match probability.

Availability refresh: Scheduler sends bounded provider/region jobs to SQS. Workers fetch and validate provider data, then commit newer availability and an outbox event in one database transaction. A stable job ID makes duplicate message delivery safe. The outbox dispatcher updates the search projection and invalidates affected cache entries. Search may briefly lag the database, so expose `lastUpdated` where the freshness promise matters.

Step 6: talk through failures and growth

What happens if Redis is unavailable? Use a bounded database fallback and limit load. What if the provider API fails? Keep the prior snapshot with an age indicator, retry later, and alert when the freshness limit is breached. What if the LLM times out? Return cards without the generated explanation. What if SQS redelivers after commit? The database job claim prevents duplicate business effects. What if the search index is behind? Show the last update time, and consider a database fallback for exact-ID details. What if a user attempts another person's watchlist? Reject before data access.

Track p95 latency, error rates, cache hit rate, database query time, search relevance, queue age, last successful provider refresh, DLQ counts, and explanation support quality. Scale the bottleneck you observe. A popular-title hot key may need a different response than a slow query plan; a stale catalog is a correctness issue even when latency looks excellent.

Two-minute answer to rehearse: “I would clarify hard filters, personalization, latency, and freshness first. With explicit traffic assumptions, I would start with a Node API, relational source of truth, reusable Redis cache, and a queued provider refresh worker. Search would combine exact and semantic signals, but structured filters and authorization would decide eligibility. The AI explanation would use retrieved evidence and fail gracefully. I would walk through database, cache, provider, queue, and model failures before adding more components.”

9. Project stories: turn the résumé into evidence

This chapter uses the résumé's project names and reported claims as **prompts to prepare**, not as independently verified implementation details. Speak precisely about what *you* built, what the team built, what was only a prototype, and what you would add in production. Do not repeat the résumé's private contact details when sharing this guide.

A 90-second introduction

Use this structure: your role and relevant experience → one concrete problem you solved → one choice you personally made → what you can discuss further. A customizable outline is: “I am a Node.js and TypeScript backend engineer with several years of experience across APIs and data-heavy workflows. One example I can explain in depth is [real project and problem]. I owned [specific action], and we observed [verified result]. I am especially interested in [relevant part of this role].” Replace brackets with your own facts; do not memorize a sentence that overstates your experience.

For each story, write a five-part card: **problem**, **constraints**, **your action**, **result and evidence**, and **learning**. Draw the request or data flow before naming tools. Practice one failure case: what could go wrong, how did the system respond, and what test or metric demonstrated it?

UnBound X: founding backend ownership

The résumé describes identity, social, learning, and market APIs, including an Alpaca integration. Choose one actual end-to-end flow. Say who sends the request, which fields you validate, how the caller is authenticated, what they are authorized to see, where state lives, what the external service does, and what the response promises. “Founding ownership” is clearer when tied to a decision and a tradeoff you personally made.

For a market API, possible *design concerns* include timeouts, provider rate limits, webhook authenticity, duplicate events, and reconciliation. Name only protections you actually implemented as past work; introduce the others as proposed improvements. **Mock question:** “What happened when the external market service timed out after you sent a request?” A good answer distinguishes a failed request from an unknown outcome and explains how your actual system handled it.

eVitalRx: the 40% and the team of 14+

The résumé reports a 40% database-query performance improvement and leadership of 14+ developers. Prepare the exact metric, baseline workload, before/after numbers, data volume, and how you measured them. If it was query latency, $(\text{before} - \text{after}) / \text{before} \times 100$ gives the percentage reduction. Do not quietly substitute a throughput gain or full-application speedup for a query metric.

Explain how you found the bottleneck: slow-query log, query plan, N+1 pattern, missing index, unnecessary rows, or another real observation. Explain your personal change and correctness check. An index might speed reads while increasing storage and write work; a batch may reduce round trips while increasing query complexity. For leadership, give one concrete review, delegation, mentorship, incident, or coordination example. “Led 14+” alone does not tell the interviewer whether they were direct reports.

Mock question: “What did the query plan show before and after?” If you no longer have the exact plan, state what you remember and what evidence you would consult; do not invent a column or timing.

Aubergine: reliability and a visible result

Choose one Node API improvement you can trace from an observed problem to a tested fix. Was it input validation, a slow dependency, a rejected promise, a database query, or something else? Explain how you reproduced it, what change you made, what the caller saw during failure, and what measurement or check confirmed improvement. Timeouts, bounded retries, and structured request IDs are good concepts to discuss, but attribute to the shipped project only those you truly used.

Mock question: “How did you know your change improved reliability instead of merely moving the error?” Answer using a specific failure case, before/after behavior, and evidence.

Incident Commander AI: approval and uncertain external outcomes

The résumé lists Python, FastAPI, MCP, Temporal, SQLite, retries, approval gates, dependency graphs, five fault scenarios, and 14 tests. Draw one actual workflow: incident input → proposed action → review/approval → execution → result. Explain which component stores approval and how it binds to the exact action and parameters. A language model may suggest an action; it must not grant itself permission to execute one.

Suppose an external action succeeds, then the process crashes before recording success. Retrying a durable workflow does not make arbitrary external effects exactly once. Use a stable operation identifier with an external idempotency mechanism if available, or reconcile the external state before retrying. If neither is safe, require intervention. Be ready to name the actual five fault scenarios and what the tests prove; do not assume this exact scenario was implemented merely because the project has tests.

Mock question: “How would a rejected approval prevent a later worker from executing an older proposal?” Explain the state transition you actually implemented, then any production safeguard you would add.

Kirana Ops Agent: the last inventory item

The résumé describes Telegram inventory, GST and Khata flows, an immutable ledger, oversell protection, idempotency, and 68 tests. Follow one sale through request ID, validation, stock change, ledger event, and response. Then trace two concurrent buyers for the last item. An in-memory check is not a database guarantee across workers; a transaction with a conditional stock update or appropriate locking protects the invariant. Only a successful stock decrement should create the sale and corresponding ledger record in the same safe boundary.

If a client loses the response and retries, a durable idempotency key bound to the request payload can return the original outcome. Reusing the same key with different input should be rejected. An immutable ledger preserves corrections as new compensating entries rather than silently rewriting history. Test count alone does not prove correctness: explain a concurrency test and a crash/retry test that matter. Do not claim tax or accounting compliance merely because GST fields exist.

Mock question: “Show the exact boundary that prevents two buyers from both seeing the last item as available.” Distinguish your project's actual mechanism from a stronger design you would choose today.

Multiplayer Chess: the server owns the board

The résumé lists TypeScript, Node, React, WebSockets, server-side move validation, reconnect, and persisted reconstruction. Draw a move command with game ID, player identity, move intent, command ID, and expected game version. The server checks membership, turn, legality, and expected version, then commits one transition and broadcasts the authoritative result. Two tabs making different moves from the same old version cannot both become the next accepted history.

On reconnect, the client asks for an authoritative snapshot or sequenced replay. Its old local board must not overwrite the server. WebSocket reconnection restores transport; storage and version rules restore correctness. Explain the persistence format you really used and a test for duplicate, stale, or unauthorized moves.

Mock question: “The server accepted a move, but the acknowledgment was lost. What happens when the client sends the same command again?” A stable command ID can return the earlier accepted result without making a second move, if that behavior was designed and stored.

10. Three timed mock interviews for a friend to run

The interviewer should read each **prompt** first and keep the coaching notes hidden until the candidate has made a genuine attempt. Ask the listed follow-ups only after the initial answer. For each mock, give one point for each of the five criteria in the four scored rounds: 20 points total. Afterward, record one strength, one correctness gap, and one communication improvement. A candidate can be honest about an unknown detail and still earn credit for a sound method.

Mock 1 - coding and backend fundamentals (60 minutes)

Round 1: introduction and ownership (5 minutes)

Prompt: “Give a 90-second introduction. Then describe one backend decision you personally owned.”

Follow-ups: “What was the alternative?” “How did you know it worked?”

Listen for, one point each: concise introduction; relevant backend example; clear personal contribution; reasoned tradeoff; evidence or an honest limit of memory. **Coach's note:** encourage a specific request/data flow rather than a stack of technology names.

Round 2: reverse a linked list (20 minutes)

Prompt: “In JavaScript or TypeScript, reverse a singly linked list in place and return the new head. Assume an acyclic list. Explain, code, and test.”

Follow-ups: “What if the list is empty?” “Why save the next pointer before changing the link?” “What does recursive reversal cost?”

Listen for: clarifies input; saves and reverses pointers correctly; returns `prev` as the new head; $O(n)$ time and $O(1)$ iterative extra space; dry-runs empty and multiple nodes. **Coach's note:** if stuck, ask the candidate to draw `prev`, `curr`, and `next` over `1 → 2 → 3`.

Round 3: Node under load (15 minutes)

Prompt: “An endpoint calls three independent services, each taking about 200 ms. It also runs a large synchronous CPU calculation. Why is it slow? How would you improve it safely?”

Follow-ups: “What if one call fails?” “Does `Promise.all` cancel the others?” “Would making the function `async` move the CPU work off the event loop?”

Listen for: overlaps independent I/O; identifies CPU blocking separately; explains `Promise.all` failure and cancellation accurately; adds timeouts and concurrency limits; chooses explicit partial-failure response behavior. **Coach's note:** “three calls take about 200 ms together” assumes they really are independent and resources are available.

Round 4: Redis cache (15 minutes)

Prompt: “A popular movie-details endpoint repeatedly reads the same database row. Design a Redis cache. What happens when Redis is unavailable?”

Follow-ups: “What is the exact key?” “How stale can the movie be?” “What if 1,000 callers miss together?” “Can update-then-delete still return old data?”

Listen for: correct hit/miss path; key scope includes region/version; TTL and invalidation tradeoff; stampede mitigation; bounded fallback and monitoring. **Coach's note:** the strongest answer names the stale-refill race without claiming TTL creates perfect consistency.

Feedback (5 minutes)

Ask the candidate which answer felt weakest. Score `/20`. Write one exact retry exercise, such as “reverse an empty and three-node list on paper” or “trace Redis outage under 1,000 requests.” Repeat that answer tomorrow without notes.

Mock 2 - movie discovery, AWS, and RAG (60 minutes)

Round 1: clarify the product (5 minutes)

Prompt: “Design an API for ‘Find a dark thriller under 100 minutes available on my services in India.’ What would you clarify first?”

Reveal if asked: assume 100,000 daily active users and 10 searches per user per day; availability may be a few hours old if the response discloses freshness. These are fictional exercise inputs. One million daily searches average about 11.6 per second; choose a peak multiplier rather than sizing to the average.

Listen for: hard versus soft requirements; region and subscription scope; latency/freshness target; explicit scale assumptions; average versus peak distinction.

Round 2: draw a design that can grow (20 minutes)

Prompt: “Sketch the API, storage, retrieval, and response path. Start with a small complete design.”

Follow-ups: “Which relationships need database constraints?” “Where can shared caching help?” “Where could it leak private data?” “What if search is down?”

Listen for: clear request/data flow; sensible records and indexes; structured eligibility filters before semantic ranking; authorization and safe caching; growth and fallback tied to measured bottlenecks. **Hint if stuck:** API → identity and filters → candidate retrieval → ranking → result.

Round 3: scheduled refresh (15 minutes)

Prompt: “Refresh provider availability every six hours. A worker can crash, a message can arrive twice, and the provider can rate-limit requests. Design the job flow.”

Follow-ups: “The database commits and the worker crashes before acknowledging-what happens?” “Which DLQ handles a Scheduler delivery failure? Which handles worker failures?”

Listen for: separates scheduling from processing; stable idempotency key and atomic effect; commit/ack failure window; bounded retries/concurrency; freshness and queue/DLQ monitoring. **Hint if stuck:** Scheduler → SQS → bounded worker.

Round 4: grounded explanation (15 minutes)

Prompt: “Add a ‘Why this movie matches’ explanation. Describe ingestion, query-time retrieval, and how you would tell whether the answer is reliable.”

Follow-ups: “Can embeddings enforce 100 minutes?” “What if a retrieved description says to ignore instructions?” “What if no evidence supports a match?”

Listen for: both RAG pipelines; exact constraints and access filters; evidence-linked claims; untrusted-source defense and abstention; evaluation of retrieval, citations, latency, and no-answer cases.

Feedback (5 minutes)

Ask for the design's biggest limitation and one simplification. Score /20. Choose one diagram to redraw from memory tomorrow. Reward a specific tradeoff, such as stale availability or expensive explanations, rather than the number of services named.

Mock 3 - project defense and debugging (60 minutes)**Round 1: eVitalRx result and leadership (10 minutes)**

Prompt: “Your résumé reports 40% better database-query performance and leadership of 14+ developers. Explain one improvement, its measurement, and your personal contribution.”

Follow-ups: “What was the baseline workload?” “What did diagnostic evidence show?” “How did you review or roll out the risky change?”

Listen for: defines the 40% metric; diagnostic evidence; personal technical action; correctness and tradeoff check; one concrete leadership example. **Coach's note:** do not reward invented timings or reporting relationships.

Round 2: Kirana's last item (15 minutes)

Prompt: “Two customers buy the last item simultaneously. Then one response is lost and its client retries. How do stock and the ledger remain correct?”

Follow-ups: “Where is the atomic boundary?” “Can a check in Node memory prevent overselling across workers?” “What test would prove this?”

Listen for: race identified; atomic conditional stock change or proper lock; ledger and stock within a safe transaction; durable retry key bound to payload; concurrency/failure test. **Coach's note:** ask which of these safeguards the project actually implemented.

Round 3: Incident Commander uncertain outcome (15 minutes)

Prompt: “An approved remediation succeeds externally. The process crashes before recording completion. On recovery, the workflow retries. What should happen?”

Follow-ups: “Which layer owns approval?” “Which layer owns the external side effect?” “What if the external API offers no idempotency key?”

Listen for: recognizes ambiguous outcome; separates durable workflow from external effect; idempotency or reconciliation; approval bound to exact action; relevant tests and honest gaps. **Coach's note:** a blanket “Temporal retries it” is incomplete.

Round 4: chess reconnect and stale commands (15 minutes)

Prompt: “A player reconnects with an old board, then two browser tabs submit different moves for the same turn. How does the server keep one valid history?”

Follow-ups: “What must be checked besides move legality?” “How does the client recover missed moves?” “What survives a server restart?”

Listen for: identity, turn, and legality; expected-version or serialized commit; duplicate command handling; authoritative snapshot/replay; persistence and failure test. **Coach's note:** transport reconnection alone does not make game state correct.

Feedback (5 minutes)

Score /20. Name two strongest answers and two exact evidence gaps to verify before the real interview. Let the candidate rehearse one question for the interviewer, for example: “Which reliability or data-freshness problem would this role work on first?”

11. Quick recall cards

Cover the right side and answer in one sentence.

Prompt	One-sentence answer
Set or Map?	Set for membership; Map when a key needs an index, count, value, or group.
Two Sum order?	Look up the complement before storing the current item.
Sliding-window rule?	Move the left edge only forward until the window is valid.
Reverse a list?	Save <code>next</code> , redirect <code>curr.next</code> , then advance.
Slow/fast cycle test?	Compare node identity, not equal values.
Does <code>async</code> fix a CPU loop?	No; synchronous JavaScript still blocks the event loop.
Why not trust a TypeScript assertion?	Runtime JSON is untrusted and still needs validation.
Redis source of truth here?	PostgreSQL; cache copies are rebuildable and may expire.
Redis invalidation race?	An older reader can refill stale data after a newer write deletes the key.
Is an SQS message exactly once?	No; design the business operation to tolerate duplicate delivery.
Scheduler DLQ versus worker DLQ?	One covers delivery into the target; the other covers processing after SQS accepted it.
What is an embedding?	A numeric representation used for semantic similarity, not a factual match probability.
Can vectors enforce runtime/region?	No; use structured filters and access checks.
What if RAG evidence is missing?	State insufficient evidence instead of inventing a fact.
What does a citation prove?	Only that the source ID exists; verify that it supports the particular claim.

Prompt	One-sentence answer
Design answer first step?	Clarify users, hard constraints, scale, latency, freshness, and failure behavior.
Strong project story?	Problem, constraints, your action, measured result, and learning.

12. Optional videos and final practice

These videos are optional introductions. The guide and active practice should be enough to work through the concepts; pause a video and explain the idea without looking before moving on.

- [What is RAG? - Apna College](#) - the example video shared for this preparation. After watching, draw ingestion and query pipelines separately.
- [Node.js Ultimate Beginner's Guide - Fireship](#) - quick runtime orientation; follow with the Node chapter here for failure and concurrency reasoning.
- [What the heck is the event loop anyway? - Philip Roberts, JSConf EU](#) - visual intuition; it uses a browser model, so do not treat every callback phase as identical to Node.
- [Redis in 100 Seconds - Fireship](#) - overview before practicing TTL, invalidation, and outage behavior.
- [AWS SQS Overview for Beginners - Be A Better Dev](#) - queue intuition; afterward, trace a crash after database commit.
- [What is Retrieval-Augmented Generation? - IBM Technology](#) - another visual explanation if retrieval versus model memory is still unclear.
- [System Design for Beginners - freeCodeCamp / Gaurav Sen](#) - optional longer lesson; then redraw the hypothetical movie design from memory.

For a final 30-minute rehearsal, solve Two Sum or linked-list reversal without notes (10 minutes), explain the Redis or AWS failure trace (5 minutes), draw the RAG pipelines (5 minutes), and give the two-minute system-design answer (5 minutes). Use the last five minutes to name one real résumé example and the evidence you can defend. If you cannot explain a point simply, return to that section and practice aloud once more.